

ResNet1D Model

Layer & Parameter Reference

Automatic Modulation Classification | PyTorch | RadioML 2016.10a / 2018.01a

Input (B, 2, L)	->	Stem (B, 64, L/4)	->	Layer 1 (B, 64, L/4)	->	Layer 2 (B, 128, L/8)	->	Layer 3 (B, 256, L/16)	->	AvgPool (B, 256, 1)	->	Output (B, 24)
-----------------	----	-------------------	----	----------------------	----	-----------------------	----	------------------------	----	---------------------	----	----------------

This document is a layer-by-layer reference for the **ResNet1D** and **BasicBlock1D** classes defined in **resnet1d.py** (PyTorch). It covers both classes in full: the residual block that forms the core building unit, the stem that processes raw I/Q input, the three residual stages, and the global pooling head. Every constructor argument, every layer inside each block, and every line of the forward pass is explained. A comparison table at the end contrasts this model with the CNN-LSTM and Keras CNN1D guides.

Contents

1. The residual connection — the key idea of ResNet
 2. BasicBlock1D — class signature & constructor arguments
 3. BasicBlock1D — layers: conv1, bn1, relu, dropout, conv2, bn2
 4. BasicBlock1D — the shortcut branch
 5. BasicBlock1D — forward(): main path + skip + final relu
 6. ResNet1D — class signature & constructor arguments
 7. Stem — the entry convolutional block
 8. `_make_layer()` — how the three stages are assembled
 9. Layer 1, 2, 3 — the three residual stages
 10. Classification head — AdaptiveAvgPool1d, Flatten, Dropout, Linear
 11. ResNet1D forward() — full data flow
 12. Model comparison table
 13. Full model code
-

1. The residual connection — the key idea of ResNet

Why skip connections exist and what problem they solve

In a plain deep network, each layer must learn to map its input to a useful output from scratch. As networks grow deeper, two problems emerge. First, gradients from the loss function shrink as they propagate backwards through many layers (vanishing gradients), making early layers learn very slowly. Second, adding more layers sometimes hurts accuracy even on training data — the network struggles to learn identity mappings when they are optimal.

ResNet (He et al., 2015) solves both problems with a single idea: instead of learning the full mapping $F(x)$, each block learns the *residual* $F(x) = H(x) - x$, where $H(x)$ is the desired output. The block then adds the original input x back: $\text{output} = F(x) + x$. If the optimal transformation is close to the identity, the block only needs to push $F(x)$ toward zero, which is much easier than learning the identity from scratch. The addition also creates a direct gradient highway back to early layers, solving the vanishing gradient problem.

```

      x (input)
      |      \
    [main path] [shortcut]
conv1->bn1->relu
  dropout
conv2->bn2      |
      |          |
      |          |
out + identity <-
      |
      relu
      |
      output

```

The shortcut branch runs in parallel with the main convolution path. If input and output shapes match, the shortcut is an identity (`nn.Identity()` — literally just `x`). If shapes differ (different channels or stride), a `1x1` convolution projects `x` to the right shape before adding.

2. BasicBlock1D — class signature & constructor arguments

The fundamental building block repeated across all three stages

```

class BasicBlock1D(nn.Module):
    expansion = 1

    def __init__(
        self,
        in_channels: int,
        out_channels: int,
        stride: int = 1,
        dropout: float = 0.1,
    ) -> None:
        super().__init__()

```

`expansion = 1` is a class-level constant inherited from the ResNet convention. In the original paper, larger variants (ResNet-50, 101) use a Bottleneck block where the final layer expands channels by a factor of 4. This BasicBlock uses `expansion = 1`, meaning output channels equal the specified `out_channels` with no expansion. The constant is not used internally here but is kept for API compatibility if you later mix block types.

Constructor parameters

<code>in_channels</code>	set by <code>_make_layer</code>	Number of input channels the block receives. Tracked by <code>ResNet1D.in_channels</code> and updated after every block so each block knows its input depth.
---------------------------------	------------------------------------	--

out_channels	64 / 128 / 256	Number of output channels after both convolutions. Determines how many features this block produces. Doubles across stages (64 -> 128 -> 256).
stride	1 or 2	Applied to the first Conv1d only. stride=1 preserves sequence length. stride=2 halves it, downsampling the temporal resolution when transitioning between stages. Only the first block of each stage uses stride=2; subsequent blocks use stride=1.
dropout	0.1	Dropout rate applied once inside the main path, after the first relu. Very light at 0.1 — ResNets generally need less dropout than plain CNNs because batch normalisation already provides regularisation.

3. BasicBlock1D — layers in the main path

conv1, bn1, relu, dropout, conv2, bn2 — explained individually

(B, C _in, L)	->	(B, C _out , L')	->	(B, C _out , L')	->	(B, C _out , L')	->	(B, C _out , L')	->	(B, C _out , L')	->	(B, C _out , L')
---------------------	----	------------------------	----	------------------------	----	------------------------	----	------------------------	----	------------------------	----	------------------------

```

self.conv1 = nn.Conv1d(in_channels, out_channels, kernel_size=3,
                        stride=stride, padding=1, bias=False)
self.bn1   = nn.BatchNorm1d(out_channels)
self.relu  = nn.ReLU(inplace=True)
self.dropout = nn.Dropout(dropout)
self.conv2 = nn.Conv1d(out_channels, out_channels, kernel_size=3,
                        padding=1, bias=False)
self.bn2   = nn.BatchNorm1d(out_channels)

```

conv1 — first convolution

A 1-D convolution with `kernel_size=3` and `padding=1`. For `stride=1` this is 'same' padding: output length equals input length. For `stride=2` (first block of each new stage) the output length halves. `bias=False` is intentional — when BatchNormalization follows, the bias is redundant because BN has its own learnable shift parameter (beta), so removing the bias saves parameters without any loss in expressiveness.

bn1 — batch normalisation after conv1

Normalises over the batch and length axes for each of the `out_channels` channels. Placed after the convolution and before ReLU (pre-activation style). This is the standard ordering in ResNets and stabilises the distribution of activations before the non-linearity fires.

relu — shared ReLU instance

Note that `self.relu` is a single module object reused in two places in `forward()` — after `bn1` and after the final addition. This is safe because ReLU has no learnable parameters and `inplace=True` operates on whatever tensor is passed to it. Reusing the object saves a tiny amount of memory.

dropout — light regularisation mid-block

Applied after the first `relu` and before `conv2`. At rate 0.1 it is very mild — only 10% of activations are zeroed. Deeper in the residual stack, the skip connection guarantees that gradient always has a direct path even if the main path neurons are dropped, so heavy dropout here would destabilise training.

conv2 — second convolution

Same kernel size and padding as `conv1`, but `stride=1` always (stride is only used in `conv1` to control downsampling). Maps `out_channels` -> `out_channels`. Together with `conv1`, this gives the block an effective receptive field of 5 timesteps at this resolution before the shortcut is added.

bn2 — batch normalisation after conv2

Placed after `conv2` but *before* the residual addition and the second `relu`. This is important: normalising before the addition keeps the magnitude of the residual term controlled. If BN were placed after the addition it would also normalise the shortcut signal, potentially interfering with the gradient highway.

4. BasicBlock1D — the shortcut branch

Identity when shapes match, 1x1 projection when they differ

```
if stride != 1 or in_channels != out_channels:
    self.shortcut = nn.Sequential(
        nn.Conv1d(in_channels, out_channels, kernel_size=1,
                  stride=stride, bias=False),
        nn.BatchNorm1d(out_channels),
    )
else:
    self.shortcut = nn.Identity()
```

The shortcut must produce a tensor with the same shape as the main path output so that element-wise addition is valid. Two conditions can cause a shape mismatch:

Condition 1: `stride != 1`. When `stride=2`, the main path halves the sequence length. The shortcut must also halve it, which is done by applying the 1x1 `Conv1d` with `stride=2`.

Condition 2: `in_channels != out_channels`. When a new stage begins, the channel count increases (e.g. 64 -> 128). The shortcut must project the input from the old channel depth to the new one. A `kernel_size=1` convolution (also called a pointwise convolution) performs this channel projection without looking at neighbouring timesteps — it simply learns a linear

combination of the input channels at each position.

When neither condition holds (same stride and same channels), the shortcut is `nn.Identity()` — a no-op module that returns its input unchanged. No parameters, no computation, zero cost.

Scenario	stride	in_ch == out_ch	Shortcut used
Same stage, same block	1	Yes	<code>nn.Identity()</code> (free pass)
New stage begins	2	No	<code>Conv1d(k=1, s=2) + BN</code>
Same stage, new depth	1	No	<code>Conv1d(k=1, s=1) + BN</code>

5. BasicBlock1D — forward()

Main path + skip addition + final activation

```
def forward(self, x: torch.Tensor) -> torch.Tensor:
    identity = self.shortcut(x)  # save (possibly projected) skip

    out = self.conv1(x)          # main path starts
    out = self.bn1(out)
    out = self.relu(out)
    out = self.dropout(out)
    out = self.conv2(out)
    out = self.bn2(out)          # main path ends (no relu yet)

    out = out + identity         # residual addition
    return self.relu(out)       # relu AFTER addition
```

Line 1: `identity = self.shortcut(x)`

Compute the shortcut branch first and store it. This ensures `x` is not accidentally modified before the addition. If shortcut is `nn.Identity()`, this is a free tensor reference. If it is a `Conv1d` projection, this is where that computation happens.

Lines 2-7: `conv1 -> bn1 -> relu -> dropout -> conv2 -> bn2`

The main path. Note that `bn2` is the last operation — there is deliberately no `relu` after `bn2`. The activation is deferred until after the addition so that both the residual signal and the shortcut signal are summed before any non-linearity clips them.

Line 8: `out = out + identity`

Element-wise addition. Both tensors must be identical in shape: `(B, out_channels, L')`. This is where the gradient highway is created. During backpropagation, the gradient of the loss with respect to `x` flows through this addition as if the main path did not exist, reaching earlier layers undiluted.

Line 9: `return self.relu(out)`

The shared relu instance is called a second time on the merged tensor. This is the post-addition activation. It ensures the output of the block is non-negative before being passed to the next block or stage.

6. ResNet1D — class signature & constructor arguments

The top-level model that assembles stem + three stages + head

```
class ResNet1D(nn.Module):
    def __init__(
        self,
        num_classes: int = 24,
        input_channels: int = 2,
        base_channels: int = 64,
        layers: tuple[int, int, int] = (2, 2, 2),
        dropout: float = 0.1,
    ) -> None:
        super().__init__()
        self.in_channels = base_channels
```

`self.in_channels = base_channels` initialises a mutable state variable that `_make_layer()` uses to track the current channel depth as it builds each stage. It starts at `base_channels` (64) and is updated in-place as each stage is created. This pattern lets the same helper method work for any depth without having to pass the previous stage's channel count explicitly.

Constructor parameters

num_classes	24	Number of output neurons (modulation classes). 24 for RadioML 2018.01a, 11 for 2016.10a.
input_channels	2	Always 2 for I/Q signals. The stem Conv1d takes this as <code>in_channels</code> .
base_channels	64	Channel depth of Stage 1. Stages 2 and 3 use <code>base_channels * 2 = 128</code> and <code>base_channels * 4 = 256</code> respectively. Increasing <code>base_channels</code> scales the entire model width uniformly.
layers	(2, 2, 2)	Number of BasicBlock1D blocks in each of the three stages. (2, 2, 2) gives a small 6-block ResNet. Standard ImageNet variants use (2,2,2,2) for ResNet-18 or (3,4,6,3) for ResNet-34. Increasing these numbers deepens the model.

dropout	0.1	Passed down to every BasicBlock1D and also applied once in the classification head. Kept light because BN already regularises effectively; heavy dropout in residual networks can destabilise the skip connections.
----------------	-----	---

7. Stem — the entry convolutional block

Rapidly compresses the raw I/Q sequence before the residual stages begin

(B, 2, L)	->	(B, 64, L/2)	->	(B, 64, L/2)	->	(B, 64, L/2)	->	(B, 64, L/4)
-----------	----	--------------	----	--------------	----	--------------	----	--------------

```
self.stem = nn.Sequential(
    nn.Conv1d(input_channels, base_channels, kernel_size=7,
              stride=2, padding=3, bias=False),
    nn.BatchNorm1d(base_channels),
    nn.ReLU(inplace=True),
    nn.MaxPool1d(kernel_size=3, stride=2, padding=1),
)
```

Conv1d(2, 64, kernel_size=7, stride=2, padding=3)

A wide-kernel convolution at the entry point. `kernel_size=7` gives the stem a large receptive field (7 timesteps) to capture coarse signal structure — carrier frequency, signal envelope, gross modulation shape — before the residual blocks refine the representation. `stride=2` immediately halves the sequence length, reducing computation for all subsequent layers. `padding=3` is exactly `kernel_size // 2`, giving 'same' padding at stride 1; at stride 2 the output length is `ceil(L / 2)`.

BatchNorm1d + ReLU

Standard BN-then-ReLU after the stem convolution. No dropout here — the stem is a single layer and dropout this early would discard raw signal information before the residual blocks have a chance to process it.

MaxPool1d(kernel_size=3, stride=2, padding=1)

A second halving of the sequence length. Together with the stride-2 stem conv, the input is compressed 4x before the first residual stage. For `L=1024` (RadioML 2018) the stem output is 256 timesteps; for `L=128` (RadioML 2016) it is 32 timesteps. `padding=1` with `stride=2` on a kernel of 3 gives output length `floor((L + 1) / 2)` — the same 'same-at-stride-2' formula used in the stem conv.

8. `_make_layer()` — assembling the residual stages

Factory method that builds a sequence of `BasicBlock1D` modules

```
def _make_layer(
    self, out_channels: int, blocks: int,
    stride: int, dropout: float
) -> nn.Sequential:
    layers = [BasicBlock1D(
        self.in_channels, out_channels,
        stride=stride, dropout=dropout
    )]
    self.in_channels = out_channels # update for next call
    for _ in range(1, blocks):
        layers.append(BasicBlock1D(
            self.in_channels, out_channels, dropout=dropout
        ))
    return nn.Sequential(*layers)
```

The first block in the list receives the special treatment: it uses the given `stride` (which may be 2 to downsample) and handles the channel transition from `self.in_channels` to `out_channels`. Its shortcut will therefore be a projection `Conv1d`.

After constructing the first block, `self.in_channels` is updated to `out_channels`. All subsequent blocks in the loop receive matching input and output channels with `stride=1`, so their shortcuts are `nn.Identity()`. This is a side effect on `self`, which is why the three `_make_layer` calls in `__init__` must happen in order — each one leaves `self.in_channels` at the value the next one needs.

9. Layer 1, Layer 2, Layer 3 — the three residual stages

Each stage doubles channel depth and (except Stage 1) halves length

```
self.layer1 = self._make_layer(base_channels, layers[0], stride=1, dropout=dropout)
self.layer2 = self._make_layer(base_channels * 2, layers[1], stride=2, dropout=dropout)
self.layer3 = self._make_layer(base_channels * 4, layers[2], stride=2, dropout=dropout)
```

Stage	out_channels	stride	Shortcut in block 1	blocks	Sequence length (L=128 e
Stem	64	2+2	n/a (not a block)	—	32 after MaxPool
Layer 1	64	1	<code>nn.Identity()</code>	2	32 (unchanged)
Layer 2	128	2	<code>Conv1d(k=1,s=2) + BN</code>	2	16

Stage	out_channels	stride	Shortcut in block 1	blocks	Sequence length (L=128 e
Layer 3	256	2	Conv1d(k=1,s=2) + BN	2	8
Head	—	—	—	—	1 (after AdaptiveAvgPool1d)

Layer 1 (stride=1): No spatial downsampling. The two blocks refine the 64-channel features from the stem without changing sequence length. Both blocks use `nn.Identity()` shortcuts since `in_channels == out_channels` and `stride == 1`.

Layer 2 (stride=2): The first block downsamples (`stride=2`) and doubles channels to 128. Its shortcut is a 1x1 Conv1d with `stride=2` + BN. The second block is shape-preserving with an identity shortcut.

Layer 3 (stride=2): Same pattern as Layer 2 but doubles to 256 channels. After this stage, the tensor is $(B, 256, L/16)$. For $L=128$ this is $(B, 256, 8)$.

10. Classification head

AdaptiveAvgPool1d + Flatten + Dropout + Linear

$(B, 256, 8) \rightarrow (B, 256, 1) \rightarrow (B, 256) \rightarrow (B, 256) \rightarrow (B, 24)$

```
self.head = nn.Sequential(
    nn.AdaptiveAvgPool1d(1),
    nn.Flatten(),
    nn.Dropout(dropout),
    nn.Linear(base_channels * 4, num_classes),
)
```

nn.AdaptiveAvgPool1d(1)

Averages the entire remaining sequence down to exactly 1 output position per channel. The argument 1 is the *target output length*, not a kernel size — PyTorch computes the kernel and stride automatically from the input length. For an input of length 8 it acts like `AvgPool1d(kernel_size=8)`. For length 64 it becomes `AvgPool1d(kernel_size=64)`. This means the model accepts any input length L without modification — the head always receives a $(B, 256, 1)$ tensor regardless of how long the original signal was. This is the key advantage over `Flatten`, which ties you to a fixed L .

nn.Flatten()

Collapses $(B, 256, 1)$ to $(B, 256)$ by removing the trailing length dimension of size 1. Equivalent to `x.squeeze(-1)` or `x.view(B, -1)`. Required because `nn.Linear` expects a 2-D input.

nn.Dropout(dropout)

Applied to the 256-d vector just before the final projection. Only one dropout in the head — light regularisation immediately before the decision boundary. Consistent with the 0.1 rate used inside each BasicBlock1D.

nn.Linear(base_channels * 4, num_classes)

`base_channels * 4 = 256` matches the output channels of Layer 3. `num_classes = 24`. Raw logits output — no softmax. Use with `nn.CrossEntropyLoss()` during training.

11. ResNet1D forward() — full data flow

Five lines that route data through stem, three stages, and head

```
def forward(self, x: torch.Tensor) -> torch.Tensor:
    x = self.stem(x)      # (B, 2, L) -> (B, 64, L/4)
    x = self.layer1(x)    # (B, 64, L/4) -> (B, 64, L/4)
    x = self.layer2(x)    # (B, 64, L/4) -> (B, 128, L/8)
    x = self.layer3(x)    # (B, 128, L/8) -> (B, 256, L/16)
    return self.head(x)   # (B, 256, L/16) -> (B, 24)
```

Step 1: `x = self.stem(x)`

The four-layer stem (Conv1d + BN + ReLU + MaxPool1d) compresses the raw signal from 2 channels to 64 and from L timesteps to L/4. This is the most aggressive compression in the network — 4x spatial reduction in a single sequential block, before any residual connections.

Step 2: `x = self.layer1(x)`

Two BasicBlock1D modules with stride=1 and 64 channels. Both blocks have identity shortcuts. The sequence length is unchanged; the 64-channel features are refined.

Step 3: `x = self.layer2(x)`

Two blocks, first with stride=2 (halves length) and a projection shortcut, second with stride=1 and identity shortcut. Channel depth doubles from 64 to 128.

Step 4: `x = self.layer3(x)`

Same structure as layer2 but channels go from 128 to 256. Sequence length halves again. After this step the tensor carries the richest, most abstract features.

Step 5: `return self.head(x)`

AdaptiveAvgPool1d collapses the sequence to 1, Flatten removes the trailing dimension, Dropout regularises, Linear projects to class logits. The entire spatial dimension is gone — the model outputs one 24-d vector per signal.

12. Model comparison table

ResNet1D vs CNN-LSTM vs Keras CNN1D across all three AMC models

Topic	Keras CNN1D	CNN-LSTM	ResNet1D
Framework	Keras / TF	PyTorch	PyTorch
Core idea	Stacked convolutions	CNN + BiLSTM	Residual skip connections
Skip connections	None	None	Every BasicBlock
Temporal modelling	Local only	LSTM over CNN features	Deep stacked blocks
Pooling	MaxPool + Flatten	MaxPool + mean(dim=1)	MaxPool + AdaptiveAvgPool1d
Output layer	Dense + Softmax	Linear (logits)	Linear (logits)
Dropout rate	0.5 (head only)	0.4 (conv + head)	0.1 (mid-block + head)
Variable len input	No	Yes	Yes
Approx params	~800 K	~1.1 M	~1.5 M
Best for	Quick baseline	Low SNR, long range	Highest overall accuracy

13. Full model code

resnet1d.py — reproduced verbatim

BasicBlock1D

```

class BasicBlock1D(nn.Module):
    expansion = 1

    def __init__(self, in_channels, out_channels, stride=1, dropout=0.1):
        super().__init__()
        self.conv1 = nn.Conv1d(in_channels, out_channels, 3,
                                stride=stride, padding=1, bias=False)
        self.bn1 = nn.BatchNorm1d(out_channels)
        self.relu = nn.ReLU(inplace=True)
        self.dropout = nn.Dropout(dropout)
        self.conv2 = nn.Conv1d(out_channels, out_channels, 3, padding=1, bias=False)
    else)
        self.bn2 = nn.BatchNorm1d(out_channels)
        if stride != 1 or in_channels != out_channels:
            self.shortcut = nn.Sequential(
                nn.Conv1d(in_channels, out_channels, 1, stride=stride, bias=False)
            ),
            nn.BatchNorm1d(out_channels),
        )
        else:
            self.shortcut = nn.Identity()

    def forward(self, x):
        identity = self.shortcut(x)
        out = self.bn2(self.conv2(
            self.dropout(self.relu(self.bn1(self.conv1(x))))))
        return self.relu(out + identity)

```

ResNet1D

```

class ResNet1D(nn.Module):
    def __init__(self, num_classes=24, input_channels=2,
                  base_channels=64, layers=(2,2,2), dropout=0.1):
        super().__init__()
        self.in_channels = base_channels
        self.stem = nn.Sequential(
            nn.Conv1d(input_channels, base_channels, 7,
                      stride=2, padding=3, bias=False),
            nn.BatchNorm1d(base_channels), nn.ReLU(inplace=True),
            nn.MaxPool1d(3, stride=2, padding=1),
        )
        self.layer1 = self._make_layer(base_channels, layers[0], 1, dropout)
        self.layer2 = self._make_layer(base_channels * 2, layers[1], 2, dropout)
        self.layer3 = self._make_layer(base_channels * 4, layers[2], 2, dropout)
        self.head = nn.Sequential(
            nn.AdaptiveAvgPool1d(1), nn.Flatten(),
            nn.Dropout(dropout), nn.Linear(base_channels * 4, num_classes),
        )

    def _make_layer(self, out_channels, blocks, stride, dropout):
        layers = [BasicBlock1D(self.in_channels, out_channels, stride, dropout)]
        self.in_channels = out_channels
        for _ in range(1, blocks):
            layers.append(BasicBlock1D(self.in_channels, out_channels,
                                       dropout=dropout))
        return nn.Sequential(*layers)

    def forward(self, x):
        x = self.stem(x)
        x = self.layer1(x)
        x = self.layer2(x)
        x = self.layer3(x)
        return self.head(x)

```